02 Variablen und Datentypen

Variablen und Datentypen

Vorlesung 12.10. und 13.10.2026
Quellen: 02_Variablen und Datentypen Teil 1.pdf (Variablen, numerische Datentypen), 02_Variablen und Datentypen_2.pdf (weitere Datentypen)

Variablen Syntax und Konventionen

Vorlesung 12.10.2026 – „Variablen und numerische Datentypen“
Themen: Variablen (Syntax, Konventionen) · numerische Datentypen · weitere Datentypen (char, String, boolean, Operatoren, Arrays; Vorlesung 13.10.2026)

Deklaration von Variablen und Datentypen

- Eine **Variable** ist ein **Platzhalter (Zwischenspeicher)** für einen beliebigen Wert in einem Programm.
- Technisch zeigt sie auf einen **konkreten Speicherbereich im Hauptspeicher**, in dem ein Wert eines bestimmten Typs abgelegt werden kann.
- Eine Variable besteht aus:
  - **Name**: eindeutiger Identifikator
  - **Datentyp**: bestimmt die **Speicherdarstellung** der Werte
- In **statisch typisierten** Sprachen wie Java sind **immer beide** notwendig.

Primitive Datentypen in Java

Primitive Datentypen (Basisdatentypen): fest definierte Typen, die in die Sprache integriert sind und nicht verändert werden können.

Typ	Werte
`int`, `byte`, `short`, `long`	positive oder negative Ganzzahl
`double`, `float`	Gleitkommazahlen
`char`	einzelnes Zeichen (Buchstabe, Ziffer oder Sonderzeichen)
`String`	Text beliebiger Länge, im Kern eine Aneinanderreihung von Zeichen
`boolean`	Wahrheitswert (`true` oder `false`)

Groß-/Kleinschreibung
`String` wird **groß** geschrieben, alle anderen **klein**.
(Streng genommen ist `String` eine Klasse; die Folie führt ihn unter den Basisdatentypen auf, vgl. .)

Deklaration, Zuweisung, Initialisierung

```
int max;          // Deklaration: Name + fester Datentyp
max = 100;        // Wertzuweisung (an zuvor definierte Variable)
int max = 100;    // Initialisierung: Deklaration + Zuweisung in einem Schritt
```

Syntaktische Regeln für Variablennamen

- Große und kleine Buchstaben, Dezimalziffern und Unterstrich.
- Das erste Zeichen darf **keine Ziffer** sein.
- **Keine Leerzeichen** im Namen.
- Etwa **fünfzig reservierte Wörter** dürfen nicht als Bezeichner verwendet werden (z.B. `class`, `int`, `public`, `static`, …).

  Syntax-Regeln werden vom Compiler** geprüft.**

Konventionen für Variablennamen

Allgemeine **Coding Conventions** für Java: Empfehlungen für einheitlichere, lesbarere Programme.

- Beginnen immer mit **Kleinbuchstaben**.
- Neue Wörter im Namen beginnen mit einem Großbuchstaben (**camelCase**).
- Bekannte Akronyme dürfen mit Großbuchstaben geschrieben werden.
- **Keine Unterstriche** im Namen.
- **Sprechende** Variablennamen verwenden.
- Mehrdeutige Abkürzungen ausschreiben.
- **Englische** Begriffe verwenden.

  Prüfungsrelevant
  Conventions werden **nicht vom Compiler** geprüft, aber laut Dozentin **in der Prüfung**.

Übungen dazu: und

Java und Typsicherheit

Java ist …
- **stark typisiert** (*strongly typed*):
- keine Variable ohne festen Datentyp
- Datentyp wird bei der Definition festgelegt
- automatische Typumwandlung nur begrenzt möglich
- **statisch typisiert** (*statically typed*): Typprüfung zur **Compile-Zeit**.

Vorteile	Nachteile
stabilere Anwendungen zur Laufzeit (geringere Gefahr von Laufzeitfehlern), höhere Codequalität	Code wirkt schwerfälliger als bei (eleganten) Skriptsprachen, „Kampf mit dem Compiler“

Kernaussagen (Take-aways)

- Variablen sind Zwischenspeicher für Werte im Rahmen einer Berechnung.
- Sie erhalten bei der Deklaration einen **festen Datentyp** und einen **festen Namen**.
- Der Datentyp legt fest, **in welcher Form** die Werte im Speicher abgelegt werden.
- Über den Namen kann immer wieder auf den Wert im Speicherbereich zugegriffen werden.
- Java kennt nur wenige Basisdatentypen, fest in der Sprache verankert.

- **Implizite Typkonversion**: der Compiler wandelt automatisch um, **nur wenn kein Informationsverlust** zu befürchten ist ().
- Das Array ist der einfache Container-Datentyp, seine Größe ist nach der Initialisierung unveränderlich ().
- Java kennt die typischen arithmetischen Operatoren ().

Numerische Datentypen

Ganzzahlen

Datentyp	Bits	kleinster Wert	größter Wert	Konstanten
`int`	32	$-2^{31} = -2.147.483.648$	$2^{31} - 1 = 2.147.483.647$	`Integer.MIN_VALUE` / `Integer.MAX_VALUE`
`long`	64	$-2^{63} = -9.223.372.036.854.775.808$	$2^{63} - 1 = 9.223.372.036.854.775.807$	`Long.MIN_VALUE` / `Long.MAX_VALUE`
`short`	16	$-2^{15} = -32.768$	$2^{15} - 1 = 32.767$	`Short.MIN_VALUE` / `Short.MAX_VALUE`
`byte`	8	$-2^{7} = -128$	$2^{7} - 1 = 127$	`Byte.MIN_VALUE` / `Byte.MAX_VALUE`

Allgemein bei n Bit: -2^{n-1} bis $2^{n-1} - 1$ (Grund:).

> **Ein Zahlenwert** ohne Punkt** ist in Java automatisch ein `int`:**
> `int max = 100;`

Gleitkommazahlen

	`double`	`float`
Größe	64 Bit (doppelte Genauigkeit)	32 Bit (einfache Genauigkeit)
Wertebereich	$\pm\ 1{,}79769 \cdot 10^{308}$	$\pm\ 3{,}40282 \cdot 10^{38}$
Präzision	ca. **15** gültige Dezimalstellen	ca. **6 bis 7** gültige Dezimalstellen
Konstanten	`Double.MAX_VALUE`, `Double.MIN_VALUE`	`Float.MAX_VALUE`, `Float.MIN_VALUE`
Literal	Zahl **mit Punkt** ist automatisch `double`: `double price = 100.0;`	muss **explizit** mit `f`/`F` markiert werden: `float price = 100.0f;`

Schreibweise numerischer Literale

Gleitkommazahlen:

```
double value = 20.5;
double value = 0.0205E3;      // 0,0205 · 10^3
double value = 205000E-4;     // 205000 · 10^-4
```

Zuordnung des Datentyps:
| Literal | Typ |

—	—
`20`	`int`
`20.0`	`double`
`20d`	`double`
`20f`	`float`
`20l`	`long`
`20b`, `20s`	**nicht möglich**

Rechenoperationen: Ergebnistyp

Bei gleichen Datentypen erhält man **nicht immer** den erwarteten Typ:

```
byte   + byte   = int
short  + short  = int
int    + int    = int
long   + long   = long
float  + float  = float
double + double = double
```

Bei **unterschiedlichen** Typen wird immer der **sichere (allgemeinere)** Typ als Ergebnis gewählt:

```
byte + short  = int
byte + long   = long
int  + float  = float
long + double = double
```

> **Empfehlung fürs erste Semester**
> `byte` und `short` sind für normale Berechnungen **ungeeignet** (gedacht für Umgebungen mit knappem Speicher). Verwende insbesondere **`int` und `double`**; `long` und `float` nur in Ausnahmefällen.

Implizite Typkonversion

Findet immer statt, wenn
- ein Wert in einem bestimmten Typ **benötigt** wird,
- der Wert **nicht** in diesem Typ vorliegt **und**
- die Konversion **ohne Informationsverlust** möglich ist.

Um Informationsverlust zu beurteilen, betrachtet man die **Wertebereiche** der Typen.

Beispiele: `double price = 3;` · `int max = 3.0;` (). Zu `char` `int`: .

Zweierkomplement

- Der Wertebereich von `int`, `long`, `short` und `byte` ergibt sich aus der **Zweierkomplementdarstellung** im Speicher.
- **Zweierkomplement** = Methode zur binären Speicherung **positiver und negativer** Ganzzahlen.
- **Positive** Zahlen: normale Binärdarstellung.
- **Negative** Zahlen: arithmetische Operation auf Binärzahlen (invertieren, +1).
- Bei n Bit: Wertebereich -2^{n-1} bis $2^{n-1}-1$.

Beispiel: −5 mit n = 4 Bit (Wertebereich −8 bis 7)

Schritt	Binär
Binärdarstellung von 5	`0101`
Inversion (alle Bits umdrehen)	`1010`
Addition von 1	`+ 0001`
Darstellung von −5	`1011`

Tabelle für n = 4

Zweierkomplement	Dezimal	Zweierkomplement	Dezimal
0000	0	1000	−8
0001	1	1001	−7
0010	2	1010	−6
0011	3	1011	−5
0100	4	1100	−4
0101	5	1101	−3
0110	6	1110	−2
0111	7	1111	−1

Muster
Das **höchste Bit** zeigt das Vorzeichen (1 = negativ). Nach 0111 (größter Wert) kommt beim Hochzählen 1000 (kleinster Wert).

Übung (Überlauf bei `2147483647 + 1`):

char und Zeichensätze

Der Datentyp `char`

- **Primitiver Typ** (engl. *character*), repräsentiert **einzelne** Textzeichen (Buchstaben oder Sonderzeichen).
- Literale stehen in **einfachen Anführungszeichen**.

```
char c = 'a';    // der Buchstabe a
char c = '5';    // das Zeichen 5
int i = 5;       // die Zahl 5
char c = '%';    // das Prozent-Zeichen
char c = ' ';    // das Leerzeichen
```

- Jedes Zeichen wird intern auf eine **positive Ganzzahl** abgebildet: `'a'` 97, `'5'` 53.
- Abbildung auf **16 Bit unsigned**.
- Die Abbildung ist über einen Zeichensatz standardisiert: **Unicode**.

Anmerkung zum Skript
Auf der Folie steht „0 bis 65.536“. Bei 16 Bit sind es genau $2^{16} = 65.536$ Werte, also **0 bis 65.535**.

Entwicklung von Zeichensätzen

Zeichensatz	Jahr	Eigenschaften
ASCII	1963	7 Bit, 128 Zeichen, die ersten 32 nicht druckbar
EBCDIC	1964	8 Bit, 256 Zeichen, eigenständiges Encoding, v.a. im IBM-Umfeld (Mainframes)
ISO 8859-1 bis -16	1985	8 Bit, je 256 Zeichen; die ersten 128 = ASCII. Normenreihe für alle europäischen Sprachen

- **ISO 8859-1**: u.a. deutsche Umlaute.
- **ISO 8859-15**: ISO 8859-1, unterstützt zusätzlich das **€-Zeichen**.
- Verwendung: ältere Dateien, Datenbanken, Anwendungen (z.B. ältere, meist statische Webseiten).

[02_Variablen und Datentypen_2.pdf not found]

Unicode und UTF

Unicode (1991, ursprünglich 16 Bit 65.536 Zeichen):
- Erste 128 Zeichen = ASCII, erste 256 Zeichen = ISO 8859-1.
- **Unicode 2.0** (seit 1996): erweitert auf **1.114.112 Zeichen**, Abbildung über **Surrogate Pairs**.
- Unterstützt alle gebräuchlichen Schriftsysteme und Zeichen weltweit.
- Jedes Zeichen hat einen festen Zahlenwert (**Codepunkt**).

UTF-x (*Unicode Transformation Format*) = Verfahren, um Codepunkte im Computer zu verarbeiten (speichern, übertragen, …):

Format	Abbildung der Codepunkte auf
UTF-8	je nach Bedarf **1 bis 4 Byte** (De-facto-Standard)
UTF-16	je nach Bedarf **2 oder 4 Byte**
UTF-32	immer **4 Byte**

> **Java**
> Java nutzt **intern** (`char` und `String`) **UTF-16**, zur Kommunikation nach außen (Dateien, Netzwerk) **UTF-8**.

Begriffe: Zeichensatz, Codepunkt, Encoding

Begriff	Bedeutung
Zeichensatz (Charset)	Menge typischerweise zusammengehöriger Zeichen
Codemenge	Menge numerischer Werte (Codepunkte)
Codepage	Zuordnungstabelle (Zeichen Zahl)
Encoding	Festlegung der **Speicherdarstellung** (Bytes)

Beispiel Unicode: Zeichensatz → *Codepage* → Codemenge (Codepunkte) → *Encoding UTF-8 bis 32* → Bytes.

Implizite Typkonversion int und char

Weil Zeichen eindeutigen Zahlen entsprechen, ist teilweise **implizite Konversion** zwischen `char` und `int` möglich:

```java
int i = 'i';    // i = 105
int a = 'a';    // a = 97
int v = '5';    // v = 53
char c = 97;    // c = 'a'
int j = c;      // j = 97
char x = -2;    // Fehler (negativ, nicht im Wertebereich von char)
```

Übung dazu:

String und boolean

String

- Dient zur Darstellung **beliebig langer Zeichenfolgen**.
- Zwei Varianten: **String-Literale** und die **String-Klasse** (kommt später).
- Literale stehen in **doppelten Anführungszeichen** (vgl. `char` mit einfachen:).

```java
String word = "Haus";
String text = "Geh ins Haus!";
```

Konkatenation Verkettung von String-Literalen mit dem +-Operator:

```java
String compiler = "Java-" + "Compiler";
```

Auch mit anderen Datentypen möglich (**Typkonversion in String**):

```java
int version = 25;
String text = "Wir arbeiten mit Java Version " + version;
```

boolean

- **Wahrheitswerte / Boolesche Werte** (nach George Boole).
- Wertebereich: genau **zwei Werte**:
  - `true` (trifft zu)
  - `false` (trifft nicht zu)
- **Keine gültigen Werte**: wahr/falsch oder 0/1.

```java
boolean isEmpty = true;
boolean isReady = false;
```

> **Pragmatik**
> Für „Ja/Nein“ gehört ein `boolean` her, nicht `int yes = 0; int no = 1;` (siehe 01 Einführung).

Arithmetische Operatoren

Java kennt die typischen arithmetischen Operatoren; sie erlauben die Formulierung mathematischer Algorithmen in Code.

Operator	Java	Priorität	Auswertung	Operanden
Positives Vorzeichen	`+1`	**1**	von rechts	1 (unär)
Negatives Vorzeichen	`-1`	**1**	von rechts	1 (unär)
Multiplikation	`a * b`	**2**	von links	2
Division	`a / b`	**2**	von links	2

Operator	Java	Priorität	Auswertung	Operanden
Modulo	`a % b`	**2**	von links	2
Addition	`a + b`	**3**	von links	2
Subtraktion	`a - b`	**3**	von links	2

Lesehilfe
Kleinere Zahl = **höhere Priorität** (Vorzeichen vor `* / %` vor `+ -`). Bei gleicher Priorität entscheidet die Reihenfolge der Auswertung (links bzw. rechts).

Ganzzahlige Division vs. normale Division

Der Operator `/` verhält sich **je nach Typ der Operanden** unterschiedlich:

Fall	Operanden	Ergebnis
Ganzzahlige Division	beide `int`	Rest wird **abgeschnitten**: `5 / 2  2`
Normale Division	beide `double` (oder `float`)	`5.0 / 2.0  2.5`

Ganzzahlige Division und Modulo

Oft **kombiniert** verwendet:
- `x / y` → Wie oft passt `y` **vollständig** in `x`?
- `x % y` → Was **bleibt** danach noch übrig (Rest)?

Aufgaben dazu: und

Zusammenhang mit Gauß
`boundary * (boundary + 1) / 2` aus 01 Einführung: erst Multiplikation, dann Division (gleiche Priorität, von links).

Arrays

Array als Container-Datentyp

- In Java (und den meisten Sprachen) **vordefinierte Container-Datenstruktur**.
- Verwaltet **Werte gleichen Typs**; die einzelnen Werte sind austauschbar.
- Zugriff über einen **festen Index**.

Beispiel: Array mit fünf `int`-Werten

Index	0	1	2	3	4
Element	20	3	345	-24	0

Eigenschaften

- **Unveränderliche Länge** (maximale Anzahl an Werten).
- **Unveränderlicher Typ** der verwalteten Werte (nach der Deklaration nicht mehr änderbar).
- Länge und Typ werden bei der **Initialisierung** festgelegt.
- Gültiger Index: $0 \leq \text{index} < \text{Länge}$
- Ungültiger Index **Laufzeitfehler** (`ArrayIndexOutOfBoundsException`).

Arbeiten mit Arrays

Definition

```
int[] intArray;
```

Zuweisung eines neuen Arrays

```
intArray = new int[4];   // Array für vier Werte
intArray = new int[0];   // Array für 0 Werte
```

Initialisierung

```
int[] intArray = new int[5];
```

Array-Literale

```
int[] intArray = new int[]{20, 3, -334, 0, -1};
int[] intArray = new int[]{};
int[] intArray = {1, 2, 3, 4, 5};
```

Zugriff

```
intArray[0] = 4;                          // Einfügen / Ändern eines Wertes
int value  = intArray[0];                 // ersten Wert lesen
int length = intArray.length;             // Länge (Anzahl der Werte)
int last   = intArray[intArray.length - 1]; // letzten Wert lesen
```

> **Merke**
> Der letzte gültige Index ist `length - 1`, nicht `length`. Die Länge ist ein Attribut (`.length`), keine Methode (keine Klammern).

> **Verwendung bisher**
> Schon in 01 Einführung kommt ein Array vor: `String[] args` in `main`, mit `args[0]` als erstem Kommandozeilenargument.

Take-away

Das Array ist der einfache Container-Datentyp zur Verwaltung von Wertemengen; seine Größe wird bei der Initialisierung festgelegt und ist unveränderlich.

Übungsaufgaben

> **Lösungen sind eingeklappt (Pfeil anklicken). Wo die Folie keine Lösung zeigt, ist es meine eigene, bitte gegenprüfen.**

Übung Variablendeklaration

Welche Deklarationen sind syntaktisch nicht korrekt (Compiler-Fehler)? ()

```
int Boundary1;      integer boundary2;   float 1st;
String first name;  double int;          String course01;
int class;          int size;            int accountNr;
int java;           double accountbalance;
char Char;          String string;       string String;   long Long;
```

> **Lösung**
> **Fehler:**

- `integer boundary2;` – `integer` ist kein Typ (richtig: `int`)
- `float 1st;` – Name beginnt mit Ziffer
- `String first name;` – Leerzeichen im Namen
- `double int;` – `int` ist reserviertes Wort
- `int class;` – `class` ist reserviertes Wort
- `string String;` – `string` (klein) ist kein Typ, `String` ist groß

Korrekt, auch wenn teils unschön: `Boundary1`, `course01`, `size`, `accountNr`, `java` (kein reserviertes Wort), `accountbalance`, `char Char`, `String string`, `long Long` (Java unterscheidet Groß/Klein).

Übung Namenskonventionen

Ok oder Verbesserungsvorschlag? ()

Beschreibung	Deklaration
Obergrenze bei Gauss	`int n;`
Postleitzahl	`String plz;`
Kontostand	`double accountbalance;`
Kursname	`String course;`
Kontonummer	`int account;`
Seitenlänge Quadrat	`double a;`
Größe/Volumen eines Pakets	`double value;`
x-Koordinate	`double x;`
Zustand ready	`boolean is_ready;`

Lösung (nach Folie, sinngemäß)
- `n` → `boundary` (sprechend)
- `plz` → `postCode` (englisch), `plz` ist „etwas unschön, da deutsch“, darf aber bleiben
- `accountbalance` → `accountBalance` (camelCase), oder `balance`
- `course` → `courseName`
- `account` → `accountNumber` bzw. `accountNr`
- `a` → `length`
- `value` → `volume`
- `x` → `xCoordinate` oder lassen
- `is_ready` → `isReady` (kein Unterstrich)

Übung Zweierkomplement

1. Tabelle für n = 4 vervollständigen: siehe .
2. Was ergibt `2147483647 + 1`? Logisch lösen.

Lösung
`2147483647` ist `Integer.MAX_VALUE` = `0111…1`. Plus 1 ergibt `1000…0`, im Zweierkomplement der **kleinste** Wert: **−2.147.483.648** (Überlauf, kein Fehler).

Übung Typkonversion

Wo tritt kein Informationsverlust auf (Anweisung korrekt)? ()
1. `double price = 3;`
2. `int max = 3.0;`

3. `float value = 3;`
4. `int degree = 3f;`
5. `int number = 3l;`
6. `long seconds = 3;`

Lösung (nach Wertebereich)
1, 3, 6 (`int` → `double`/`float`/`long`: Zieltyp hat größeren Wertebereich).
2, 4, 5 (`double`/`float`/`long` → `int`: Informationsverlust möglich → Compilerfehler).

Übung char

`int sum = u + v + w + x;` mit `int u = 49; char v = '4'; int w = '4'; char x = 49;` ()
> **Lösung**
> `'4'` = 52 (da `'5'` = 53). `x` = Zeichen `'1'`, als Zahl 49. $49 + 52 + 52 + 49 =$ **202**

Übung 1 Tapete

20 m Tapete, Wand 3 m hoch: wie viele Bahnen, wie viel Verschnitt? ()
> **Lösung**
> `20 / 3` = **6 Bahnen**, `20 % 3` = **2 m Verschnitt**.

Übung 2 Sekunden

Wie vielen Minuten entsprechen 35000 Sekunden, wie viele Sekunden bleiben übrig?
> **Lösung**
> `35000 / 60` = **583 Minuten**, `35000 % 60` = **20 Sekunden**.